

Problem 1

Part 1 (b)

- Every state in the search tree is an instance of the MCTSNode class. We can keep track of the total number of visits to a node using the self.N and the accumulated value from the root player's point of view. We can keep track of the relationship between the two using self.parent and self.children.
- How I chose which action to expand was that the algorithm has a list of untried_actions for each node, which is initialized with all actions available from that node's current state. During the expansion phase, we use .pop() to select and remove an action from this list to make sure each action is expanded into a new child only once.
- We used uniform random rollout policy. The algorithm uses the call actions(state) function to identify all legal moves and uses random.choice() to select.

Part 2

Budget	Win Rate	Ave Score Diff	Avg ms/move
20	95%	18.73	0.14
200	95%	22.10	1.28
2000	93.3%	21.80	12.41
20000	88.3%	20.97	118.28

Part 3

1. Analysis

- a. The overall win rate against minimax is high, but over time it goes down as seen in the chart, going from 95% to 88.3%. But other than the slight decrease in win rate, the average score difference is positive, which means the MCTS outperforms the minimax
- b. The move computation stays the same as the budget.

2. MCTS

- a. I think that it is a little unexpected. Before doing this assignment, I thought that the minimax would always make the best moves possible to try to win the game. So the win rate being 88% - 95% was not expected.
- b. I believe that the reason that MCTS has such a high win rate is that the depth limit of the minimax starter code. Since the minimax didn't search until the end of the game, the MCTS is able to outperform it.
- c. In order to make the two algorithms have the same exact performance, you could increase the search depth of the minimax algorithm and reduce the MCTS budget to almost zero. This is how you would make both perform almost exactly the same.

3. With a large iteration budget, the MCTS might struggle due to a high variance of random rollouts. In our data, going from 200 to 20,000, the win rate decreased as the budget

grew. This is because higher budgets can cause the algorithm to over-index on a branch that performs well in random situations but is, in reality, losing a move against an opponent who is more structured.

Problem 2

Part 1 - Connectives

- a. $(P \vee Q) \wedge \neg(P \wedge Q)$, meaning exactly one of P or Q is true

Part 2 - KB definition

Running the program resulted in this:

Puzzle 1

A is a Knave

Puzzle 2

A is a Knave

B is a Knight

Puzzle 3

A is a Knight

B is a Knave

C is a Knight

Problem 3

Part 1 - Natural Language to FOL

- a. Every int has exactly 1 successor & its not itself
 $\forall x \exists y (\text{Successor}(x,y) \wedge x \neq y \wedge \forall z (\text{Successor}(x,z) \rightarrow y = z))$
- b. Every int is either even or odd, but not both
 $\forall x ((\text{Even}(x) \vee \text{Odd}(x)) \wedge \neg(\text{Even}(x) \wedge \text{Odd}(x)))$
- c. The successor of an even number is odd
 $\forall x \forall y ((\text{Even}(x) \wedge \text{Successor}(x,y)) \rightarrow \text{Odd}(y))$
- d. The successor of an odd number is even
 $\forall x \forall y ((\text{Odd}(x) \wedge \text{Successor}(x,y)) \rightarrow \text{Even}(y))$
- e. If y is the successor of x, then y is larger than x
 $\forall x \forall y (\text{Successor}(x,y) \rightarrow \text{Larger}(y,x))$
- f. The relation Larger is transitive
 $\forall x \forall y \forall z ((\text{Larger}(x,y) \wedge \text{Larger}(y,z)) \rightarrow \text{Larger}(x,z))$

Query:

For every int x, there exists an even int y that is larger than x

$\forall x \exists y (\text{Even}(y) \wedge \text{Larger}(y,x))$

AI usage was simply just an aid with bug fixing as well as general problem explanation throughout the assignment.